

VISVESVARAYA NATIONAL INSTITUTE OF TECHNOLOGY, NAGPUR

Department of Computer Science and Engineering

THESIS PROPOSAL REPORT

Constraint-Driven Automated Test Vector Generation for Avionics Bus Stimulation and LRU Validation

- **Candidate Name:** Akash Kaushal
- **Degree:** Master of Technology in Computer Science and Engineering
- **Domain:** Embedded Systems, Formal Verification, Avionics Software Engineering
- **Date:** September 2026

Page 1: Title, Abstract & Key Highlights

M.Tech 2nd Year Thesis Proposal

Topic: Constraint-Driven Automated Test Vector Generation for Avionics Bus Stimulation and LRU Validation

Student Name: Akash Kaushal

Department: Computer Science and Engineering

Degree: Master of Technology (M.Tech)

Simple Executive Abstract

Modern airplanes rely on specialized computers called **LRUs (Line Replaceable Units)** to control critical tasks like navigation, steering, and engine speed. These computers talk to each other over dedicated cables using standard aerospace protocols like **ARINC 429** and **MIL-STD-1553**. Inside each computer, a small C program called a **bus parser** reads and unpacks every incoming message.

Right now, engineers test these parsers by writing test scripts manually. This manual process takes weeks and misses strange, hard-to-imagine corner cases—such as broken parity bits, out-of-order messages, or corrupted sensor values. If an unexpected message arrives mid-flight, a poorly tested parser can crash, freeze in an infinite loop, or corrupt computer memory.

This project builds an **automated testing tool** using Python and a smart logic-solving engine called **Z3 (an SMT solver)**. Instead of guessing test cases by hand, we write down the protocol rules as mathematical constraints. The Z3 solver automatically generates thousands of tricky, malformed, and edge-case packets in seconds. We feed these packets directly into the C parser in a simulated Linux environment to verify that the software safely rejects bad data, never crashes, and recovers cleanly.

Key Highlights of the Proposal

- **The Problem:** Testing airplane computer software manually is slow and misses critical boundary bugs.
- **The Solution:** Use an automated logic solver (Z3) to mathematically craft broken test packets.

- **The Target:** The C software parser that decodes ARINC 429 and MIL-STD-1553 data.
- **The Platform:** Runs entirely on a standard Linux computer in Software-in-the-Loop (SIL) mode—no expensive aircraft hardware needed.
- **Expected Result:** Higher test coverage, automated discovery of hidden software bugs, and a faster certification workflow.

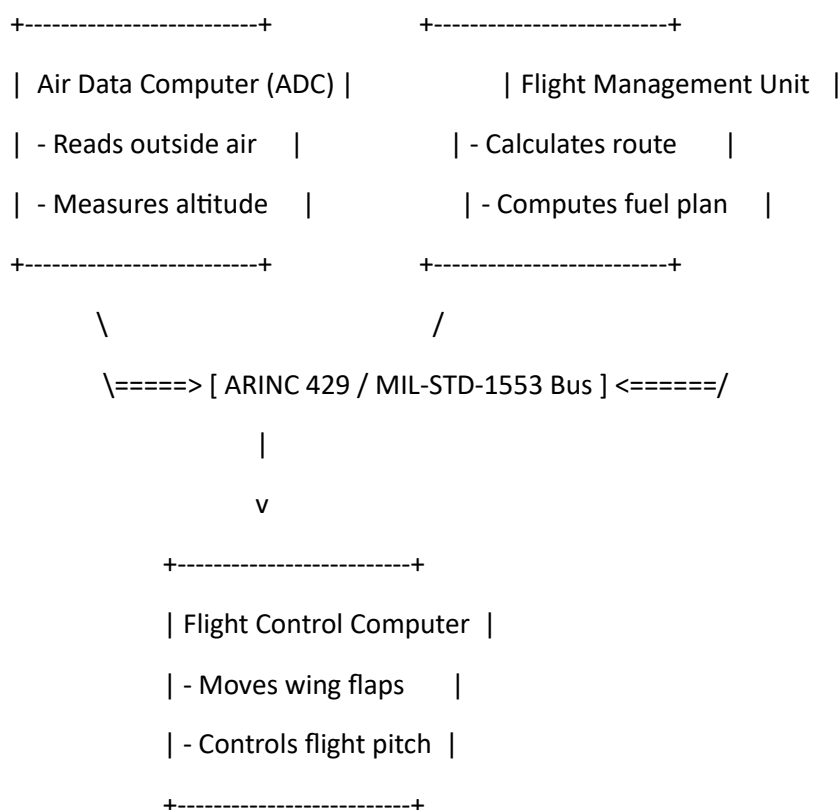
Page 2: Chapter 1 — Introduction & Background

1.1 What is an LRU?

An airplane does not have a single central computer like a laptop. Instead, it uses dozens of modular, box-shaped computers called **Line Replaceable Units (LRUs)**. Each box handles a specific job:

- **Flight Control Computer (FCC):** Moves the wing flaps and rudder.
- **Air Data Computer (ADC):** Measures altitude, airspeed, and air temperature.
- **Flight Management Unit (FMU):** Computes the flight path and fuel consumption.

These LRUs are called "Line Replaceable" because if one fails on an airport runway, a technician can unscrew the box, slide it out, and plug in a fresh replacement unit in under 30 minutes without rewiring the aircraft.



1.2 How Do LRUs Talk to Each Other?

Consumer computers use Wi-Fi or standard Ethernet, but commercial and military aircraft use dedicated, fault-tolerant serial buses:

1. **ARINC 429 (Commercial Airlines):**

- Transmits data as single **32-bit binary words**.
- One wire pair carries data in one direction only (simplex).
- Used for straightforward sensor updates (e.g., transmitting current altitude every 100 milliseconds).

2. MIL-STD-1553 (Defense & Military Aircraft):

- Uses a shared two-wire cable where multiple computers share the line.
- Controlled by a "Bus Controller" talking to "Remote Terminals."
- Transmits **20-bit words** in strict command-and-response sequences.

1.3 What is a Bus Parser?

Every LRU contains a **bus parser**—a C code function whose sole job is to grab raw 1s and 0s from the wire, check if they are valid, unpack the numbers (like altitude or heading), and pass them to the flight-control algorithms.

Page 3: Chapter 2 — The Problem & Why Current Testing Fails

2.1 The Current Industry Bottleneck: Manual Testing

In aerospace testing labs, engineers spend weeks manually writing test scripts. A human engineer looks at the specification manual and writes tests like:

- *"Send Altitude = 10,000 feet."*
- *"Send Altitude = 0 feet."*
- *"Send Altitude = 50,000 feet."*

While this works for normal flying conditions, **humans are bad at guessing millions of bizarre corrupted combinations.**

2.2 What Goes Wrong in the Real World?

During a flight, aircraft experience electrical interference, lightning strikes, sensor power fluctuations, and loose pin connections. These cause corrupted or distorted signals on the bus wires:

Normal Signal: 1 0 1 0 1 1 0 0 (Clean bits, correct format)

Corrupted Signal: 1 0 ? 0 0 1 0 0 (Flipped bits, missing fields, sudden burst)

If the C parser receives these unexpected signals, common programming bugs can emerge:

- **Unsigned Integer Wrap-around:** The code subtracts 1 from a length field of 0, creating 255, which causes the parser to read past its memory buffer.
- **State Machine Freeze (Deadlock):** The parser receives two command messages back-to-back without a data message in between. It gets stuck waiting forever for the data, ignoring all future flight commands.
- **Divide-by-Zero Exceptions:** A noisy bus packet sets a scale multiplier field to zero, crashing the CPU when the parser computes calibrated airspeed.

2.3 Why Existing Commercial Tools (like LDRA) Don't Solve This Completely

Aerospace companies use commercial testing suites like LDRA or VectorCAST. These tools are designed to test **isolated C functions** (e.g., calling `calculate_speed(x, y)` with extreme numbers).

However, they do not simulate **full streams of raw, multi-word network bus traffic** with timing jitter, parity errors, and state-machine transitions. That gap is where software crashes slip through into expensive flight tests.

Page 4: Chapter 5 — Literature Review & Existing Approaches

4.1 Comparing Testing Approaches

Testing Method	How It Works	Big Advantage	The Fatal Flaw
1. Manual Test Scripts	Engineers hand-write test values into test scripts.	Tests realistic, expected flight scenarios.	Extremely slow; only tests ~1% of possible error combinations.
2. Random Fuzzing (Blind Testing)	A script throws totally random 1s and 0s at the software.	Fast; generates millions of inputs automatically.	Useless here: avionics packets require strict parity/checksum bits. Random bits get dropped immediately at the outer filter and never test inner logic.
3. Unit Test Tools (LDRA / VectorCAST)	Generates inputs for isolated C functions inside a computer.	Great at measuring statement-by-statement code coverage.	Does not test real-time serial bus streams or multi-word communication state machines.
4. Our Approach (SMT / Z3 Solver)	Uses mathematical logic to craft packets that are 95% valid, but broken in one exact spot.	Automatically bypasses outer checks to test deep, hidden software branches.	Requires modeling the protocol rules into the solver upfront.

4.2 What is an SMT Solver (Z3)?

An **SMT (Satisfiability Modulo Theories) Solver** is a software tool developed by computer scientists (such as Microsoft Research's **Z3**) to solve logic puzzles.

Instead of writing a program that guesses values, you give Z3 a set of algebraic rules:

- *Rule 1:* Variable *X* must be a 32-bit integer.

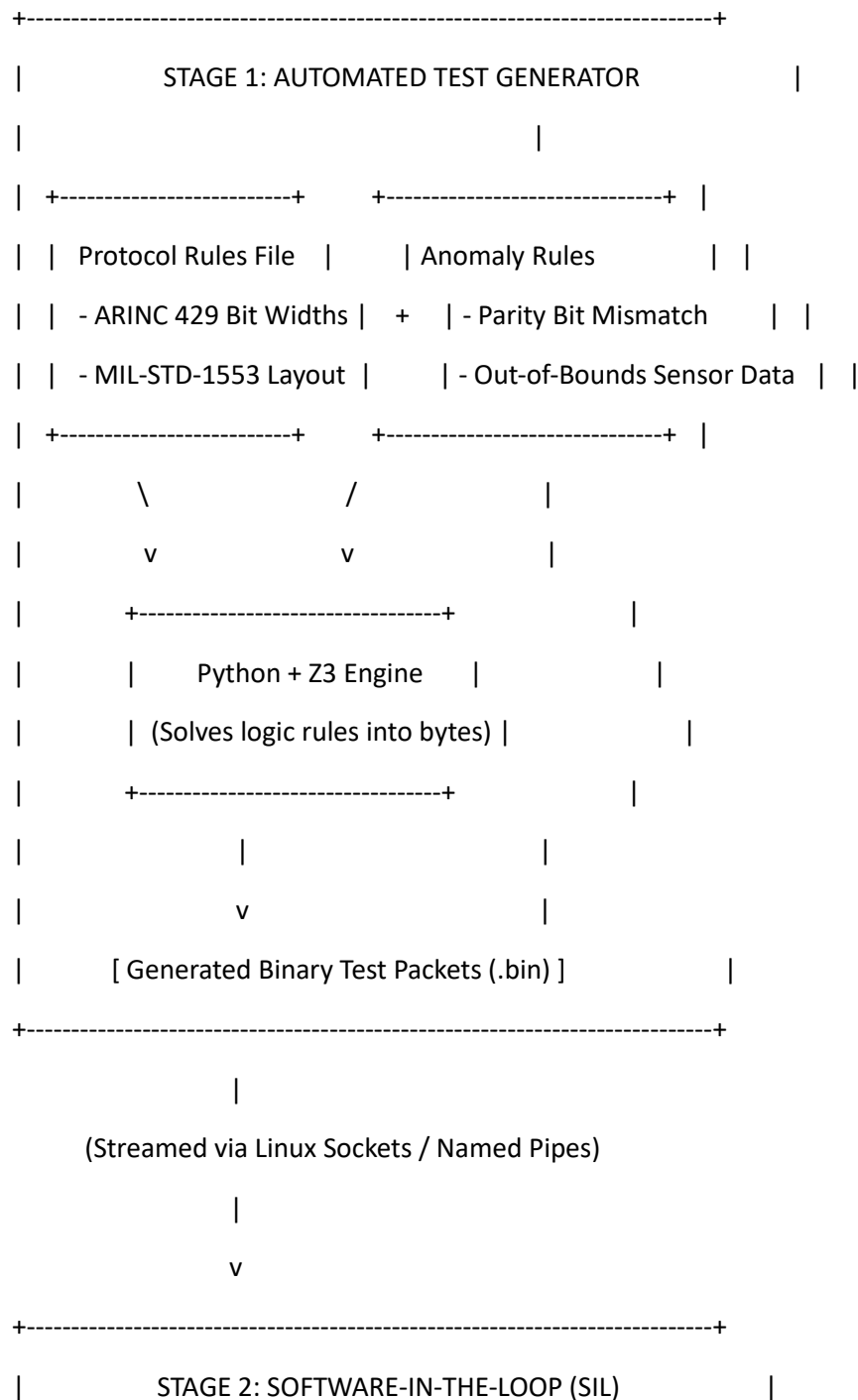
- *Rule 2:* The first 8 bits of X must equal 132 (the code for Altitude).
- *Rule 3:* The total count of 1 bits in X must be an even number (an illegal parity error).

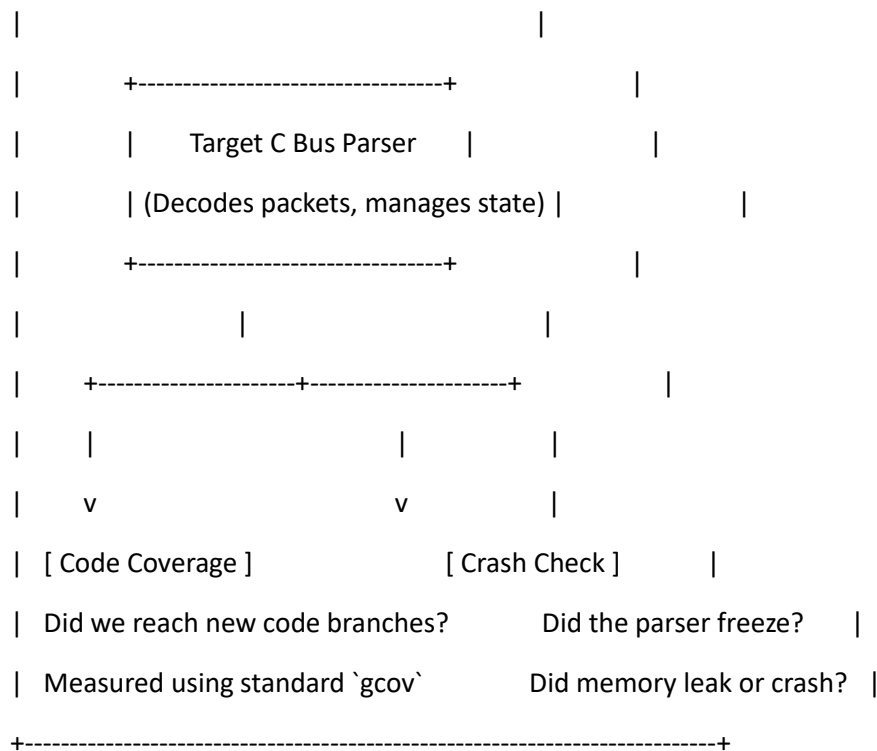
You run `solver.check()`, and Z3 calculates the exact 32-bit number that satisfies all three conditions in a fraction of a millisecond.

Page 5: Chapter 5 — Proposed System Architecture

5.1 The Complete Pipeline

The proposed system consists of two parts: the **Test Generator (Python + Z3)** and the **Test Target (C Parser on Linux)**.





5.2 The 4 Steps in Plain English

1. **Model the Protocol:** We teach Python the layout of an avionics word (where the labels, payload, and parity bits live).
2. **Synthesize Malformed Vectors:** We run our script, and Z3 generates hundreds of broken or extreme test packets.
3. **Stream into the C Parser:** A Python test runner sends these binary packets into the compiled C parser over standard Linux inter-process channels.
4. **Evaluate Results:** We inspect code coverage using Linux gcov and confirm whether the parser safely discarded corrupted packets without crashing or freezing.

Page 6: Chapter 6 — Protocol Details & How We Break Them

6.1 Understanding an ARINC 429 Word (32 Bits)

An ARINC 429 word is a single 32-bit integer divided into 5 distinct zones:

Bit: 1 ----- 8 | 9 --- 10 | 11 ----- 29 | 30 ---- 31 | 32

Field: Label | SDI | Data | SSM | Parity

Role: What data? | Which LRU | The number (e.g. altitude) | Status Flag | Odd Parity

- **Label (Bits 1–8):** Tells what kind of data is inside (e.g., 0x84 represents Altitude).
- **SDI (Bits 9–10):** Identifies which specific computer sent the message.
- **Data (Bits 11–29):** The actual number (e.g., binary representation of 32,000 feet).
- **SSM (Bits 30–31):** Sign/Status Matrix (11 = Normal Operation, 00 = Failure warning).

- **Parity (Bit 32):** An error-check bit. The total number of 1 bits across all 32 bits must always be an **ODD** number.

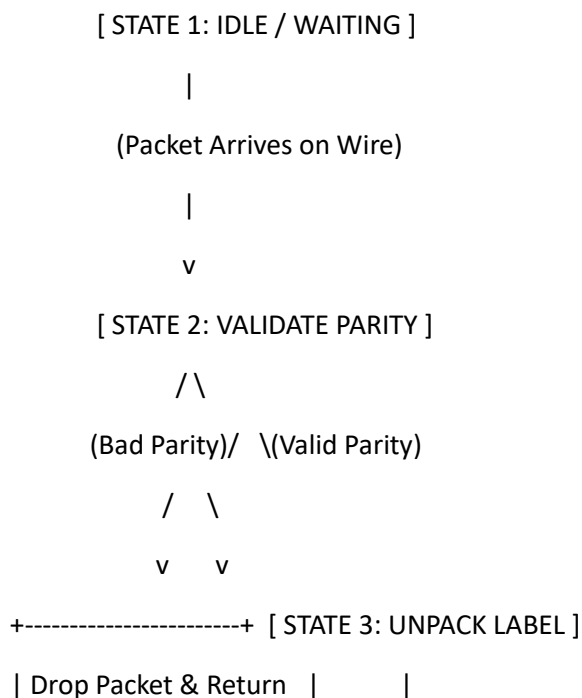
6.2 Test Scenarios Our Tool Generates Automatically

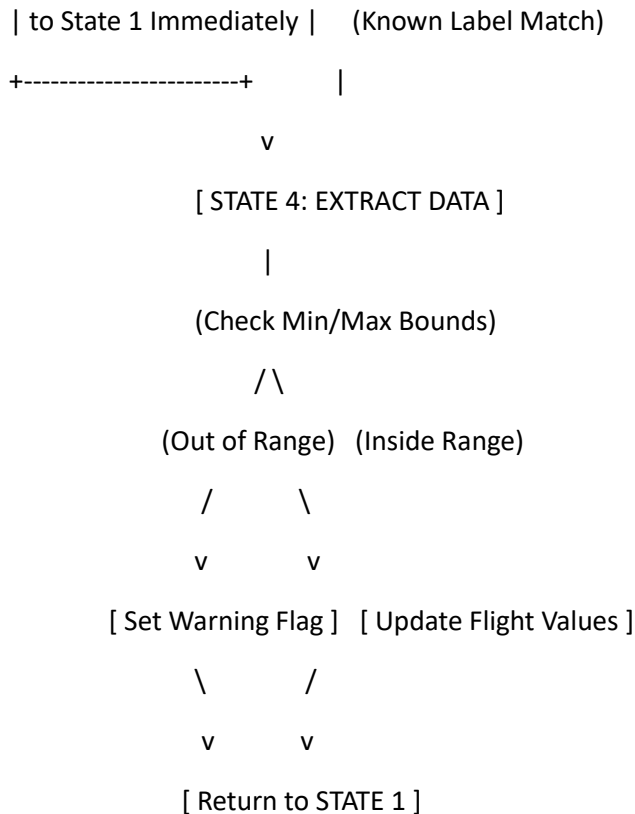
- **Scenario 1: The Parity Trick (Testing Error Filters)**
 - *What it is:* A packet has a real label (0x84) and normal status (11), but the parity bit is deliberately calculated to make the total number of 1s **EVEN**.
 - *What we test:* Does the C parser reject this corrupted packet immediately, or does it accidentally pass corrupted numbers into the flight computer?
- **Scenario 2: Max Payload Saturation (Testing Arithmetic Limits)**
 - *What it is:* A packet has correct parity and valid label, but the 19-bit data field is set to its absolute maximum limit (0x7FFFF = 524,287).
 - *What we test:* Does the C parser's math overflow or wrap around to negative numbers when converting raw bits into physical units?
- **Scenario 3: Sequence Interruption (Testing State Machines)**
 - *What it is:* In MIL-STD-1553, sending two "Send Data" commands back-to-back without waiting for the status response.
 - *What we test:* Does the parser's state machine recover, or does it lock up permanently?

Page 7: Chapter 7 — Target Parser Design & SIL Setup

7.1 The Parser State Machine

The C program we are testing follows a standard avionics state machine:





7.2 The Software-in-the-Loop (SIL) Harness

- **OS:** Standard Linux (Ubuntu).
- **Compiler:** GCC with coverage flags enabled: `gcc -fprofile-arcs -ftest-coverage parser.c -o parser`.
- **Transport:** The Python generator sends binary packets to the C parser through a Linux **Named Pipe (FIFO)** or local socket. This closely mimics a hardware serial transceiver receiving bytes over a physical cable.

Page 8: Chapter 8 — Benchmarking & Evaluation Plan

8.1 How Will We Prove This Tool Works?

To write a successful thesis and research paper, we must prove our tool performs better than older methods. We will run three side-by-side experiments on the same C parser:

Test Suite 1: Manual Testing --> 50 hand-written test packets

Test Suite 2: Random Fuzzing --> 50,000 completely random 32-bit words

Test Suite 3: Our Z3 Tool --> 500 constraint-solved edge-case packets

8.2 The 4 Metrics We Will Measure

1. **Branch Coverage Percentage (Using gcov):**
 - Measures how many if-else branches and error-handling conditions were exercised.
 - *Target:* Achieve > 90% branch coverage, including hard-to-reach error-recovery routines.

2. Number of Logic Flaws Discovered:

- Count how many buffer overflows, integer wraps, and deadlock states our tool triggers compared to manual tests.

3. Recovery Latency (Microseconds):

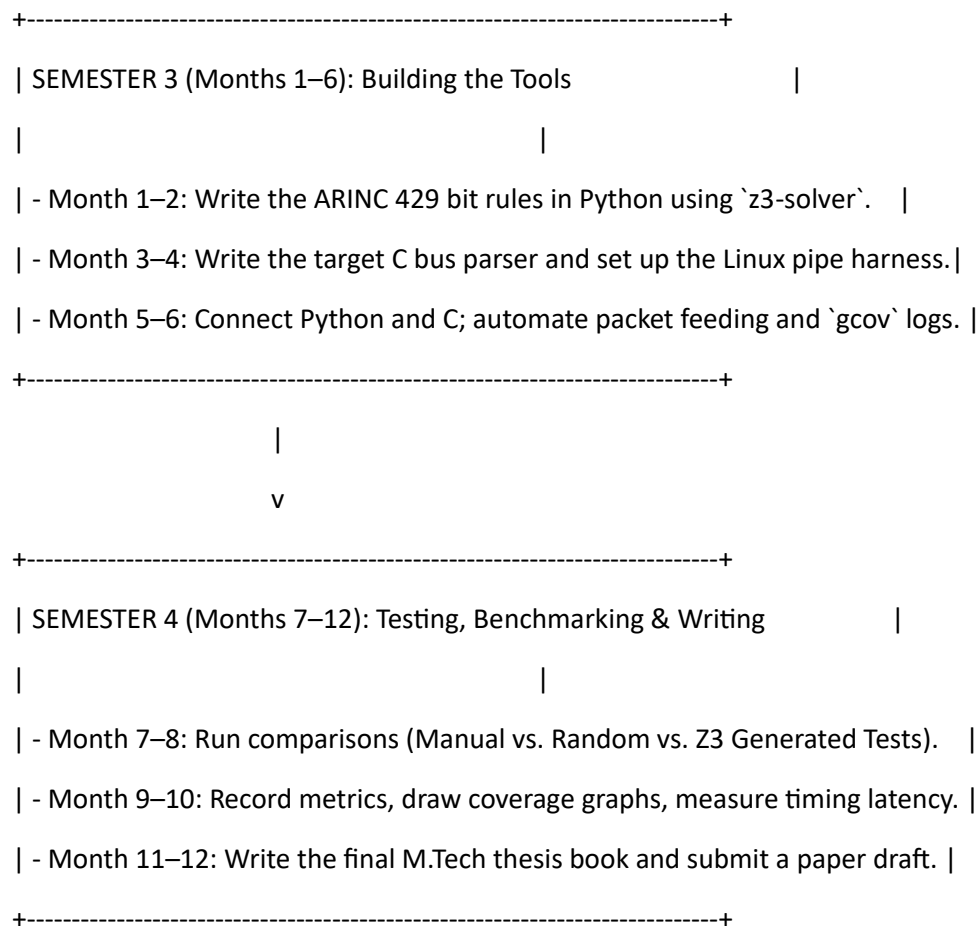
- Using high-resolution Linux timers (`clock_gettime`), we measure how many microseconds the C parser takes to reject a bad packet and return to the IDLE state.

4. Memory Stability:

- Running the parser under Linux **AddressSanitizer (ASan)** and **Valgrind** to prove zero memory leaks even after processing 1,000,000 continuous malformed packets.

Page 9: Chapter 9 — Project Roadmap & Expected Deliverables

9.1 Semester-by-Semester Work Plan



9.2 Tangible Deliverables for VNIT

1. **The Generator Tool:** A clean, documented Python repository using Z3 that outputs binary test packets for avionics protocols.
2. **The C Testbed:** An instrumented C-based parser simulating an LRU bus interface on Linux.
3. **The Experimental Results:** Clear graphs and data tables showing superior branch coverage and edge-case discovery.

4. Publications:

- The complete M.Tech 2nd-year Thesis Dissertation.
- A research paper submitted to an embedded systems or aerospace conference (such as IEEE/AIAA DASC).

Page 10: Chapter 10 — References & Standard Literature

1. **RTCA / EUROCAE.** (2011). *DO-178C: Software Considerations in Airborne Systems and Equipment Certification*. Standard aviation safety document.
2. **RTCA / EUROCAE.** (2011). *DO-333: Formal Methods Supplement to DO-178C*. Guidelines recognizing mathematical solvers like SMT for software safety evidence.
3. **Aeronautical Radio, Inc. (ARINC).** (2004). *ARINC Specification 429: Mark 33 Digital Information Transfer System (DITS)*. Official standard for the 32-bit avionics bus.
4. **United States Department of Defense.** (1978). *MIL-STD-1553B: Digital Time Division Command/Response Multiplex Data Bus*. Military aircraft bus specification.
5. **De Moura, L., & Bjørner, N.** (2008). *Z3: An Efficient SMT Solver*. Microsoft Research, TACAS Conference Proceedings.
6. **Spitzer, C. R., Ferro, U., & Spitzer, P. R.** (2015). *Digital Avionics Handbook*. Standard university reference textbook for aircraft bus architectures.
7. **Godefroid, P., Levin, M. Y., & Molnar, D.** (2012). *SAGE: Whitebox Fuzzing for Security Testing*. Communications of the ACM. (Seminal paper on combining solvers with software testing).
8. **Free Software Foundation.** (2022). *GCC gcov: A Test Coverage Program*. Official GNU documentation for measuring code coverage on Linux systems.

How to Turn This Into Your Final Submission

- If using **Microsoft Word / LibreOffice**: Copy and paste the 10 sections above directly into a blank document. Use standard margins (1 inch), set headings to Bold 14pt, and body text to 11pt Calibri or Times New Roman. Each page numbered 1 to 10 corresponds to a page in your report.
- If using **Overleaf / LaTeX**: Put each page into a `\section{...}` or `\pagebreak` block. The ASCII diagrams will format cleanly inside standard `\begin{verbatim} ... \end{verbatim}` blocks.